The Effect of AI Usage on Developer Productivity

Ábel Nátán Dankó, Alexandru Pennert & Raisa Amalia

The adoption of AI in software development is becoming increasingly widespread, with AI tools now an integral part of many developers' workflows. According to the Stack Overflow Developer Survey [1], 64.8% of developers use AI at least once per week, with 47.1% reporting daily usage across experience levels. Common applications include code generation and debugging, where AI can assist developers in completing tasks more efficiently.

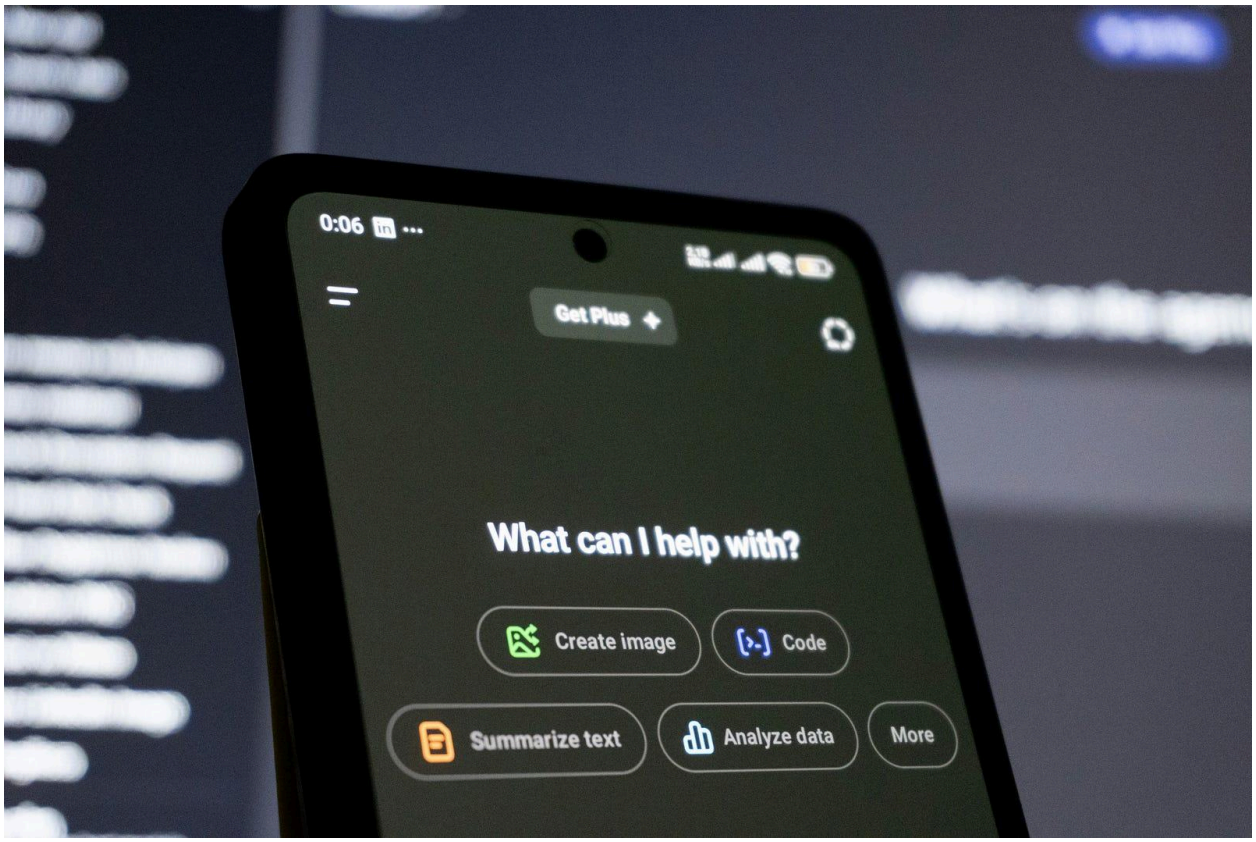

Photo by Zulfugar Karimov on Unsplash

Despite this, the impact of AI on developer productivity remains unclear. While AI may accelerate development by reducing time spent on routine tasks, it may also introduce overhead, such as reviewing and correcting generated code. This leads to the research question:

Do developers with high AI usage exhibit significantly higher productivity, measured by commit frequency and volume, compared to those with no AI usage?

Hypothesis

Based on practical experience and prior observations, we hypothesize that higher AI usage is associated with higher productivity. This is expected to be most evident in large-scale projects with structured workflows and frequent deadlines, where AI can accelerate repetitive tasks such as boilerplate generation, debugging, and documentation. By reducing routine work, AI has the potential to increase overall development output. Therefore, we expect a positive relationship between AI usage and developer productivity. This hypothesis directly informs the project's approach, which focuses on analyzing commit-level activity in active open-source repositories to evaluate differences in productivity across AI usage levels.

Identifying Relevant Projects

A key challenge in studying AI usage is reliably identifying AI-assisted code, as no widely accepted tools exist to detect AI-generated code. To address this, we identify AI usage at the commit level by analysing commit messages rather than code.

Recent AI assistants such as GitHub Copilot and Claude can automatically label commits with a "Co-authored-by" tag when AI is used. By searching for commit messages containing "Co-authored-by:" followed by "Claude" or "Copilot," we classify whether AI was used in a commit.

A limitation of this approach is that it only captures explicitly labelled usage, which can be edited or removed, making the method more susceptible to false negatives than false positives. However, this conservative approach ensures that detected AI usage is verifiable.

We used the GitHub REST API to collect commit data from open-source repositories, restricting the dataset to commits from January 1, 2025 onward, when automated labeling appears to have been introduced.

Repository selection focuses on large, active projects with structured workflows and frequent development activity, where differences in productivity are more observable. Star count is used as a proxy for project maturity and complexity, as highly starred repositories are more likely to have many contributors and regular commits.

To ensure relevance, repositories were required to have at least 100 pull requests and at least 5% AI-assisted pull requests during the study period. A pull request was classified as AI-assisted if at least one of its commits was labelled as such. These criteria ensure that both AI usage and developer activity are sufficiently present to allow meaningful comparison across usage groups. However, the selection process may introduce bias.

Thresholds were chosen to balance data quality and sample size, and different choices may yield different results. Uncertainty around when AI labeling was introduced may also result in under-detection in earlier commits.

Choice of Metrics and Purpose

Developers are classified into three groups based on AI usage: none, low, and high. High usage is defined as more than 20% of commits being AI-assisted, distinguishing substantial AI reliance while maintaining a sufficient sample size. Low usage includes 0–20%, while none represents no AI usage.

Productivity is measured using **commit frequency** and **commit volume**. Temporal metrics such as lead time and cycle time are excluded, as they depend heavily on team practices and processes, particularly code review behavior, which can distort results. Focusing on commit frequency at the individual level reduces external influences and better isolates developer activity. Programming language is also included as a contextual variable to assess whether patterns differ across environments or are influenced by language-specific factors.

However, commit frequency alone does not fully capture output, as developers vary in commit practices. Some commit frequently in small increments, while others make fewer but larger contributions. To account for this, commit volume, measured as the total number of lines added and removed, is included as a complementary metric. While imperfect, it provides additional context on the scale of contributions.

Together, these metrics provide a more comprehensive measure of productivity, capturing both activity frequency and change magnitude. They directly operationalize productivity in this study, enabling comparison across developers with different levels of AI usage.

Data Cleaning

Preprocessing

The raw dataset consists of individual commit-level observations. To construct meaningful measures of productivity, the data is aggregated at the developer-month level.

For each developer and month:

- **Commit frequency** is calculated as the total number of commits
- **Commit volume** is calculated as the sum of additions and deletions

This ensures that each observation represents a consistent unit of developer activity over time.

Cleaning

Missing values and invalid observations were removed, including entries with non-positive values or inconsistent metrics. For the language variable, missing values were replaced with "Unknown", to retain these observations in the overall dataset. However, these were excluded from language-specific analysis to maintain interpretability. These steps ensure that the dataset is internally consistent, free from invalid or misleading observations, and suitable for reliable statistical analysis.

Statistical Analysis

To compare productivity across AI usage levels, the data was summarized at the developer level. For each developer, the median monthly commit frequency and median monthly total code changes were calculated to represent typical productivity over time. This aggregation ensures that each observation represents an independent unit of analysis, avoiding bias from repeated observations of the same developer.

The median is used as the primary measure of central tendency due to the highly skewed distribution of the data. Unlike the mean, which is sensitive to extreme values, the median provides a more robust representation of typical developer behavior.

Developers were grouped into three categories based on their average AI usage:

- None (0%)
- Low (0–20%)
- High (>20%)

To evaluate whether differences in productivity across these groups are statistically significant, hypothesis testing was conducted.

Hypotheses

- **Null hypothesis (H_0):** There is no difference in productivity between developers with different levels of AI usage.
- **Alternative hypothesis (H_1):** There is a difference in productivity between developers with different levels of AI usage.

Given the highly skewed and non-normal distribution of the data, non-parametric statistical tests were used, as they do not rely on assumptions of normality. A **Kruskal-Wallis** test was

applied to assess whether there are overall differences between the three groups, followed by pairwise **Mann-Whitney U** tests to identify which specific groups differ.

Insights

The results indicate that the relationship between AI usage and developer productivity is non-linear. While statistical tests confirm significant differences between AI usage groups, the data does not support the expectation that higher AI usage leads to higher productivity. Instead, developers with low AI usage demonstrate higher productivity than non-users and high AI users, while high AI users do not outperform non-users. This contradicts the initial hypothesis that productivity would increase with higher AI usage.

At the developer level, this pattern is consistent across both commit frequency and volume changes.

AI Usage group	Commit Frequency	Volume Changes
None 0%	2.0	50.00
Low 0-20%	**17.0**	**4,675.50**
High 20%+	2.0	136.25

Median productivity metrics by AI usage groups

Mean values follow the same directional pattern but are larger than the medians, reflecting the skewed distribution of productivity metrics.

AI Usage group	Commit Frequency	Volume Changes
None 0%	4.8	8,091
Low 0-20%	**37.3**	**54,123**
High 20%+	7.6	6,278

Mean productivity metrics by AI usage groups

Therefore, median-based comparisons provide a more reliable representation of typical developer behavior.

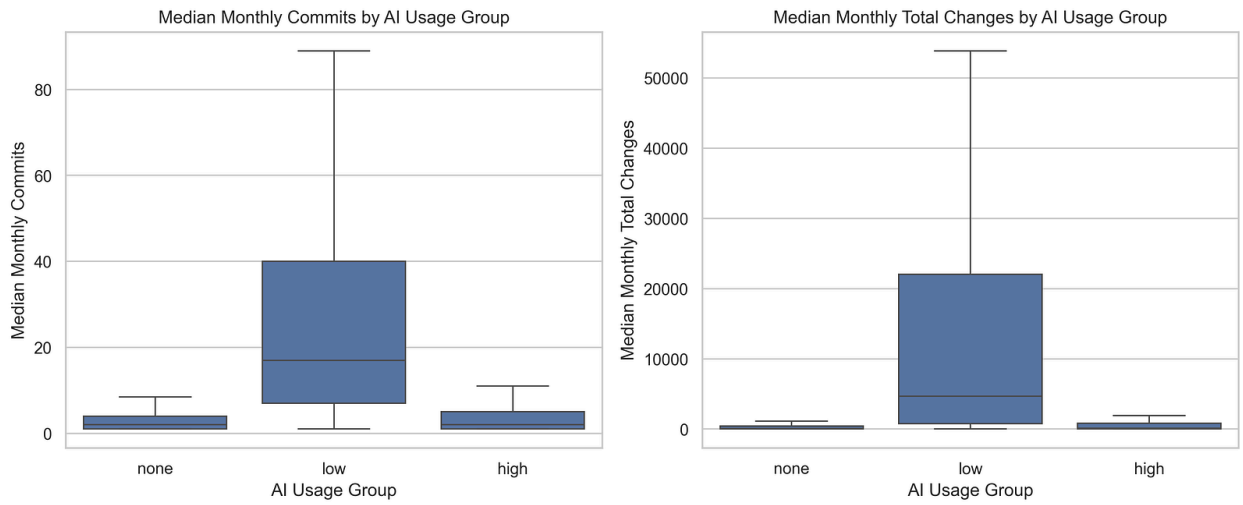

Figure 1: Median monthly commits and Median monthly total changes (volume) by AI Usage group.

Statistical significance

The Kruskal-Wallis test confirms a significant difference across groups ($p < 0.001$), with pairwise Mann-Whitney tests indicating significant differences between all group pairs.

These findings suggest that moderate AI usage may provide the greatest productivity benefits. One explanation is that developers use AI as a support tool for repetitive tasks such as debugging or boilerplate generation, improving efficiency while retaining control over higher-level design and code quality.

In contrast, high AI usage is not associated with higher productivity. Over-reliance may introduce overhead from reviewing, correcting, and integrating generated code, meaning increased output does not necessarily translate into efficient or meaningful contributions.

While the results are statistically significant, they do not imply causation. Instead, they indicate an association: moderate AI usage aligns with higher observed productivity. The null hypothesis is therefore rejected, but the relationship is not linear and does not increase with higher AI usage, suggesting an optimal level of AI use.

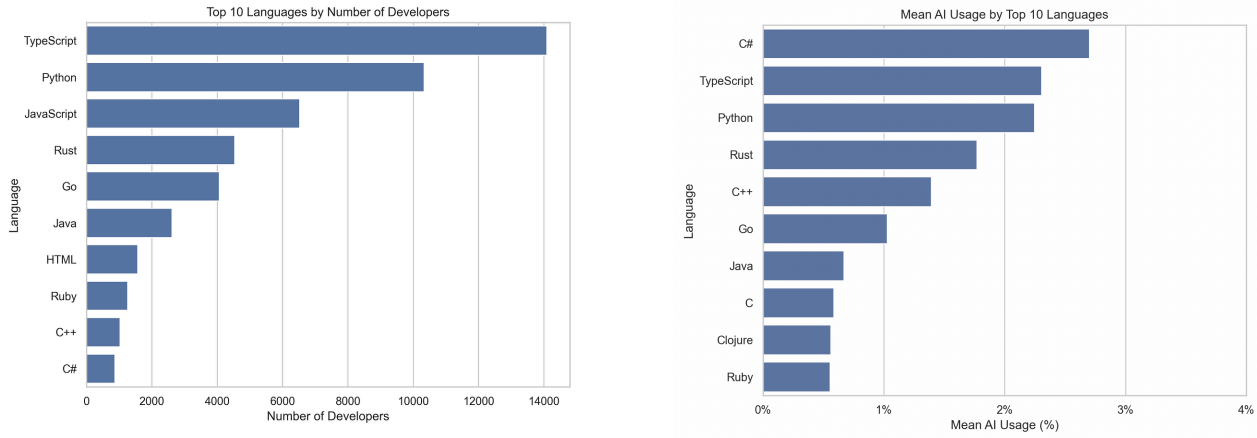

Figure 2: Top 10 most used languages and mean AI usage by top 10 languages

AI usage varies slightly across programming languages, with somewhat higher usage in Go, Rust, and TypeScript, and lower usage in Java and C++, as shown in **Figure 2**. These differences may reflect variations in developer workflows and ecosystems, as newer and more rapidly evolving languages such as Go, Rust, and TypeScript are often associated with environments that encourage earlier adoption of tools like AI assistants.

On the other hand, languages such as Java and C++ are more commonly used in established or enterprise-oriented contexts, where development practices may be more standardized and slower to incorporate new tools.

However, overall AI usage remains low across all languages (below 4% on average), and the differences are relatively small. This suggests that AI usage is less strongly influenced by the programming language itself and more by developer practices and workflows.

Limitations

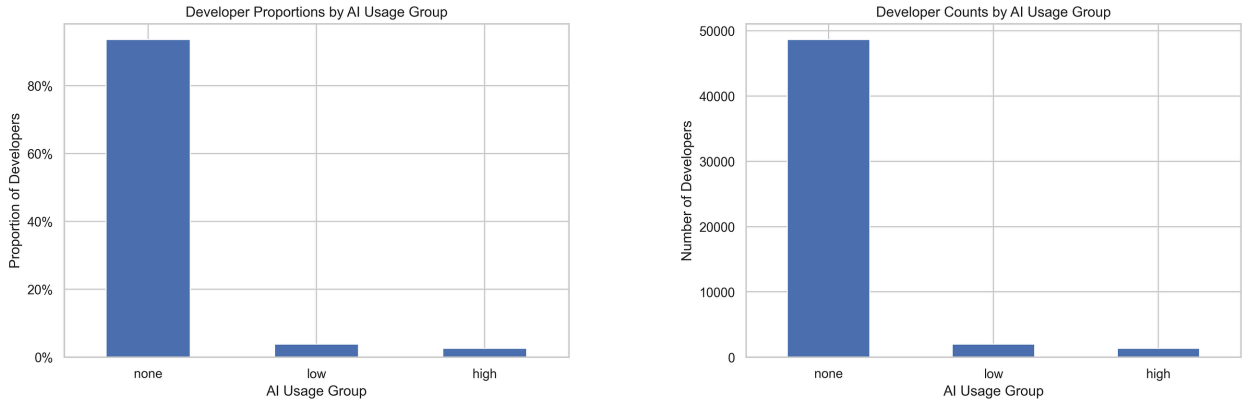

Figure 3: Developer Counts and Developer Proportions by AI Usage Group

Several limitations should be considered when interpreting the results. The dataset is highly imbalanced, as illustrated in **Figure 3,** with over 93% of developers classified as non-AI users. This may affect the stability and reliability of comparisons, particularly for the low and high usage groups.

AI usage is likely underestimated, as detection relies on commit labels that can be disabled or edited, leading to expected false negatives. Thus, some developers classified as non- or low-AI users may actually use AI more extensively.

Commit frequency and volume are imperfect productivity proxies, as commit practices and effort vary. Frequent small commits versus infrequent large ones may not reflect effort or code quality, and small changes may require substantial effort while large changes may be relatively simple.

The thresholds used to define AI usage groups and filtering criteria were selected heuristically, so different choices may lead to variations in the results.

The analysis does not account for developer experience or role. More experienced developers may produce cleaner code with fewer commits, while less experienced developers may commit more frequently, potentially influencing productivity independently of AI usage.

Finally, the dataset is limited to active open-source GitHub repositories with measurable AI-labelled commits, so the findings may not generalize to other contexts such as private repositories, smaller projects, or teams with different development and review practices.

Future Work

Future research could extend this study in several ways. Sensitivity analysis on the chosen thresholds could improve the robustness of the results. Expanding the dataset to include private or enterprise code bases and smaller or less active projects, would improve generalizability across different development contexts.

In addition, combining quantitative analysis with qualitative methods (e.g., developer surveys or interviews) could provide deeper insight into how and why developers use AI tools and the extent of review or rework required for AI-generated code. This would help explain the mechanisms behind the observed productivity patterns.

Conclusion

In conclusion, AI usage does not have a straightforward relationship with developer productivity in active open-source projects. While productivity differs significantly across AI usage groups, the relationship is not linear.

Developers with low levels of AI usage exhibit substantially higher productivity than both non-users and high AI users. This suggests that AI is used most effectively when applied selectively, while heavier reliance on AI is not associated with increased productivity.

References

[1] Stack Overflow. (n.d.). *Ai*. 2025 Stack Overflow Developer Survey. Retrieved March 1, 2026, from https://survey.stackoverflow.co/2025/ai/

Data processing: https://github.com/hipsty3/soft-anal-dev-productivity